

Homework Exercises 10

Dr. Sarvar Abdullaev
s.abdullaev@inha.uz

December 1, 2022

After completing all lab exercises below, zip all your solution files into a single archive with your student ID (e.g. u210023.zip) and upload it to eClass before the official deadline.

1 UML and Design Patterns

In this section, you should draw UML Class Diagrams and Sequence Diagrams for different software requirements, and solve some design problems using Design Patterns.

1.1 Restaurant - UML Class Diagram

The owner of a small restaurant wants a new information system to store data for all meals consumed there and also to keep a record of ingredients kept in stock. After some research he reached the following requirements list:

1. Each ingredient has a name, a measuring unit (e.g. olive oil is measured in liters, while eggs are unit based) and a quantity in stock. There are no two ingredients with the same name.
2. Each dish is composed of several ingredients in a certain quantity. An ingredient can, of course, be used in different dishes.
3. A dish has an unique name and a numeric identifier.
4. There are several tables at the restaurant. Each one of them has an unique numeric identifier and a maximum amount of people that can be seated there.
5. In each meal, several dishes are consumed at a certain table. The same dish can be eaten more than once in the same meal.
6. A meal takes place in a certain date and has a start and end time. Each meal has a responsible waiter.
7. A waiter has an unique numerical identifier, a name, an address and a phone number.
8. In some cases it is important to store information about the client that consumed the meal. A client has a tax identification number, a name and an address

As a software architect, you should design a UML Class Diagram for a software which can satisfy above requirements. Draw your UML diagram on a blank sheet of paper and save its scanned image as **restaurant.png**.

1.2 UML Sequence Diagram

Draw a UML Sequence Diagram indicating that object A calls the method `bb()` in object B. Then B performs the requested action and returns control to A, and then A calls the method `cc()` in object C. Then A should enter a loop and call `dd()` method of D object as long as `continue` variable is true. After exiting the loop, A should check if `hasResult` is true, and if yes, call `evaluate` method of `Client`. Draw your UML diagram on a blank sheet of paper and save its scanned image as **sequenceDiagram.png**.

1.3 Ice cream flavours

Create an `IceCream` class with a method `make()` that produces a vanilla ice cream. We should be able to sprinkle multiple ingredients such as `nuts`, `strawberries` and `cookies` on top of the original vanilla ice cream, as the user chooses his preferences from the menu.

Describe your design using UML class diagram and save it as `iceCreamFlavors.png`. Then implement your design in Python script `iceCreamFlavors.py`.

1.4 Starfighter and his gadgets

You have an `Starfighter` class with a method `speak()`. You should be able to equip with your starfighter with various gadgets such as `Weapon` with `attack()` method. `PlasmaGun` is a weapon and when it is used it sounds like "Phew-phew-phew!". `Blaster` is also a weapon, and when it is used it sounds like "Bah-bah-bah!". `Lightsaber` is also a weapon, and when it is used it sounds like "Swish-swish-swish". Starfighter can dynamically pick up different weapons and use them against enemies.

Describe your design using UML class diagram and save it as `starfighter.png`. Then implement your design in simple Python script `starfighter.py`.

1.5 Dog Orchestra

Imagine, there is a dog orchestra named "Good Boys" consisting of Husky, Dusky and Chappy. Every member of this band can `woof()`. But Chappy got ill, and wants to send its friend cat Tom as a replacement. Tom can only `meow()`. Unfortunately, conductor can only work with dogs. How Chappy can solve this problem?

Describe your design using UML class diagram and save it as `dogs.png`. Then implement your design in simple Python script `dogs.py`.